

SAP ADT Python — Documentação do Projeto

Versão: 0.2.0 Criado em: 02/04/2026 Atualizado em: 02/04/2026 Autor: egoetz Objetivo: Workbench de Desenvolvimento ABAP via SAP ADT REST API + Servidor MCP para Cursor IDE

1. Visão Geral

Este projeto implementa um cliente Python para a API REST do SAP ADT (ABAP Development Tools), os mesmos endpoints HTTP que o Eclipse ADT utiliza sob `/sap/bc/adt/`. Inclui um servidor MCP (Model Context Protocol) que permite ao Cursor IDE funcionar como um workbench completo de desenvolvimento ABAP.

Funcionalidades

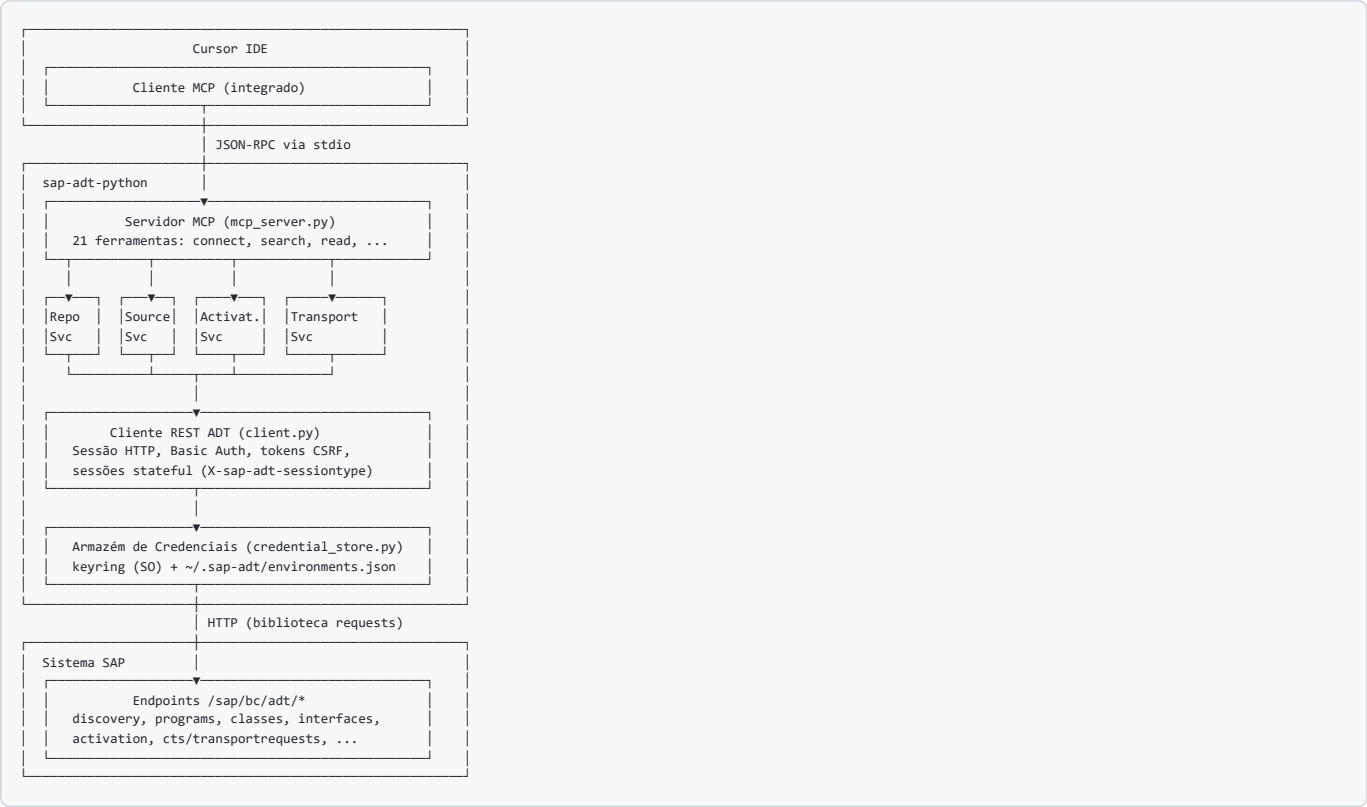
- **Conexão** com sistemas SAP via HTTP Basic Auth + gerenciamento de token CSRF
- **Armazenamento seguro de credenciais** no gerenciador de credenciais do sistema operacional (Windows Credential Manager, macOS Keychain, Linux Secret Service) via `keyring`
- **Suporte a múltiplos ambientes** — salvar e alternar entre sistemas SAP (ED2, EQ2, PRD, etc.)
- **Pesquisa** no repositório ABAP por programas, classes, interfaces, grupos de funções, tabelas
- **Navegação** em pacotes (incluindo pacotes com namespace como `/TAX/SPED_MONITOR`) e visualização de metadados de objetos
- **Leitura/Escrita** de código-fonte ABAP com gerenciamento automático de bloqueio e sessões ADT stateful
- **Leitura e gravação em arquivos locais** — exportar código-fonte ABAP para arquivos `.abap` e importar de arquivos locais
- **Ativação** (compilação) de objetos com relatório de erros
- **Verificação de sintaxe** sem ativar
- **Gerenciamento de transports** — verificar bloqueios, listar/criar/liberar transport requests
- **Criação/Exclusão** de objetos ABAP (programas, classes, interfaces)

2. Sistemas SAP Alvo

Os ambientes são armazenados de forma segura via `sap_save_password` e carregados via `sap_connect(system_id="...")`.

Parâmetro	ED2 (Desenvolvimento)	EQ2 (Qualidade)
Host	<code>awsntwpstx01.engdb.infra</code>	<code>awsntwpstx01.engdb.infra</code>
Porta	<code>8000</code>	<code>8008</code>
Protocolo	HTTP	HTTP
Mandante	<code>500</code>	<code>500</code>
Usuário	<code>egoetz</code>	<code>egoetz</code>
Idioma	<code>EN</code>	<code>EN</code>

3. Arquitetura



4. Estrutura do Projeto

```
D:\ENGDB\sap-adt-python\
├─ pyproject.toml           # Metadados e dependências do projeto Python
├─ requirements.txt         # Dependências pip
├─ .env.example             # Template de variáveis de ambiente
├─ .gitignore               # Regras de ignore do Git
├─ README.md                # Guia de início rápido
├─ AGENTS.md                # Instruções para agentes Cursor
├─ .cursor/
│   ├── mcp.json            # Configuração do servidor MCP para o Cursor
│   └─ rules/
│       └─ abap-dev.mdc     # Regra do Cursor para workflow de desenvolvimento ABAP
├─ docs/
│   ├── PROJECT_DOCUMENTATION.md # Visão geral e arquitetura do projeto (inglês)
│   ├── PROJECT_DOCUMENTATION_PT-BR.md # Este arquivo (português)
│   └─ MCP_COMMAND_REFERENCE.md  # Referência completa de comandos MCP com exemplos
├─ src/
│   └─ sap_adt/
│       ├── __init__.py      # Inicialização do pacote
│       ├── client.py        # Cliente HTTP ADT (autenticação, CSRF, sessões stateful)
│       ├── credential_store.py # Armazenamento seguro de senhas via keyring do SO
│       ├── models.py        # Modelos de dados (AbapObject, TransportRequest, etc.)
│       ├── exceptions.py    # Hierarquia de exceções customizadas
│       ├── repository.py    # Pesquisa, navegação de pacotes, metadados de objetos
│       ├── source.py        # Leitura/escrita de código-fonte com gerenciamento de bloqueio
│       ├── activation.py    # Ativação de objetos, verificação de sintaxe
│       ├── transport.py     # Gerenciamento de transport requests (CTS)
│       └─ mcp_server.py     # Servidor MCP expando 21 ferramentas
├─ scripts/
│   ├── run_mcp.py          # Inicializador do servidor MCP
│   └─ test_connection.py   # Script de teste de conexão
├─ tests/
│   ├── __init__.py
│   ├── test_client.py      # Testes unitários para client, models, resolução de URI
│   └─ test_repository_metadata.py # Testes unitários para parsing de XML de metadados
```

5. Endpoints da API REST ADT Utilizados

Todos os endpoints são relativos à URL base (ex: `http://awsntwpstx01.engdb.infra:8000`).

Autenticação e Discovery

Método	Path	Header Accept	Finalidade
GET	/sap/bc/adt/discovery	application/atomsvc+xml	Descoberta de serviços + token CSRF (via X-CSRF-Token: Fetch)

Repositório

Método	Path	Header Accept	Finalidade
GET	/sap/bc/adt/repository/informationssystem/search	application/xml	Busca rápida de objetos ABAP
POST	/sap/bc/adt/repository/nodestructure	application/vnd.sap.as+xml	Navegação na árvore de pacotes
GET	/sap/bc/adt/programs/programs/{nome}	application/vnd.sap.adt.programs.programs.v2+xml	Metadados de programa
GET	/sap/bc/adt/oo/classes/{nome}	application/vnd.sap.adt.oo.classes.v4+xml	Metadados de classe
GET	/sap/bc/adt/oo/interfaces/{nome}	application/vnd.sap.adt.oo.interfaces.v5+xml	Metadados de interface

Código-Fonte

Método	Path	Header Accept	Finalidade
GET	/sap/bc/adt/programs/programs/{nome}/source/main	text/plain	Ler código-fonte de programa
GET	/sap/bc/adt/oo/classes/{nome}/source/main	text/plain	Ler código-fonte de classe
GET	/sap/bc/adt/oo/interfaces/{nome}/source/main	text/plain	Ler código-fonte de interface
GET	/sap/bc/adt/functions/groups/{nome}/source/main	text/plain	Ler código-fonte de grupo de funções
PUT	{source_uri}?lockHandle=...&corrNr=...	text/plain	Gravar código-fonte (requer lock handle + sessão stateful)

Ciclo de Vida de Objetos

Método	Path	Header Accept	Finalidade
POST	{object_uri}?_action=LOCK&accessMode=MODIFY	application/vnd.sap.as+xml	Bloquear objeto (sessão stateful)
POST	{object_uri}?_action=UNLOCK&lockHandle={h}	application/vnd.sap.as+xml	Desbloquear objeto (sessão stateful)
POST	/sap/bc/adt/activation	application/xml	Ativar (compilar) objetos
POST	/sap/bc/adt/programs/programs	application/vnd.sap.adt.programs.programs.v2+xml	Criar novo programa
POST	/sap/bc/adt/oo/classes	application/vnd.sap.adt.oo.classes.v2+xml	Criar nova classe
POST	/sap/bc/adt/oo/interfaces	application/vnd.sap.adt.oo.interfaces.v2+xml	Criar nova interface
DELETE	{object_uri}	application/xml	Excluir objeto

Gerenciamento de Transports (CTS)

Método	Path	Header Accept	Finalidade
POST	/sap/bc/adt/cts/transportchecks	application/xml	Verificar se objeto necessita de transport
GET	/sap/bc/adt/cts/transportrequests	application/vnd.sap.adt.transportorganizer.v1+xml	Listar transport requests
POST	/sap/bc/adt/cts/transportrequests	application/xml	Criar novo transport request
POST	/sap/bc/adt/cts/transportrequests/{trkorr}/newreleasejobs	application/xml	Liberar transport

6. Fluxo de Autenticação

1. Requisição inicial para /sap/bc/adt/discovery inclui:
- Authorization: Basic <base64(usuario:senha)>

◦ X-CSRF-Token: Fetch

◦ sap-client: 500

◦ sap-language: EN
2. Resposta contém:
- Token CSRF no header X-CSRF-Token

◦ Cookies de sessão (SAP_SESSIONID_* , sap-usercontext)
3. Requisições subsequentes incluem:
- O token CSRF no header X-CSRF-Token (para POST/PUT/DELETE)

◦ Cookies de sessão (gerenciados automaticamente pelo requests.Session)
4. Na expiração do CSRF (HTTP 403), o cliente automaticamente busca um novo token e repete a requisição.
5. Ordem de resolução da senha:
1. Argumento password explícito

2. Variável de ambiente SAP_PASSWORD

3. Gerenciador de credenciais do SO via keyring (se system_id for fornecido)

7. Armazenamento Seguro de Credenciais

As credenciais são gerenciadas pela biblioteca keyring , que utiliza o gerenciador de credenciais nativo do sistema operacional:

Sistema Operacional	Backend
Windows	Windows Credential Manager
macOS	Keychain
Linux	Secret Service (GNOME Keyring, KWallet)

Estrutura de armazenamento:

- **Senhas** → armazenadas no keyring do SO sob o serviço `sap-adt-mcp`, chave `{system_id}:{usuario}`
- **Parâmetros de conexão** → armazenados em `~/.sap-adt/environments.json` (sem segredos)

Fluxo de trabalho:

```
1. sap_save_password(system_id="ED2", host="...", user="egoetz", password="****")
2. sap_connect(system_id="ED2") → senha recuperada do keyring do SO
```

8. Fluxo de Gravação de Código-Fonte

A ferramenta MCP `sap_write_source` executa o fluxo completo:

1. **Verificação de transport** — Determina se o objeto necessita de transport request; detecta automaticamente o transport de registro
2. **Bloqueio (Lock)** — Adquire um bloqueio de edição no objeto (`POST ?_action=LOCK`) com `X-sap-adt-sessiontype: stateful`
3. **Gravação (Write)** — Envia o novo código-fonte (`PUT .../source/main?lockHandle=...&corrNr=...`) com header de sessão stateful
4. **Ativação (Activate)** — Compila o objeto (`POST /activation`)
5. **Desbloqueio (Unlock)** — Libera o bloqueio (`POST ?_action=UNLOCK`) com header de sessão stateful

Se qualquer etapa falhar, o bloqueio é liberado no bloco `finally`.

Opções de entrada de código:

- `source_code` — código ABAP como texto inline
- `source_file` — caminho para um arquivo `.abap` local (usado quando `source_code` está vazio)

9. Metadados de Objetos

A ferramenta `sap_object_metadata` obtém informações detalhadas sobre um objeto ABAP usando headers `Accept` específicos do fornecedor:

Tipo de Objeto	Media Type Accept
Programa (PROG)	<code>application/vnd.sap.adt.programs.programs.v2+xml</code>
Classe (CLAS)	<code>application/vnd.sap.adt.oo.classes.v4+xml</code> (fallback: v2)
Interface (INTF)	<code>application/vnd.sap.adt.oo.interfaces.v5+xml</code> (fallback: v2)
Grupo de Funções (FUGR)	<code>application/vnd.sap.adt.functions.groups.v2+xml</code>

O parser de metadados XML trata:

- Atributos no nível raiz com namespace `adtcore:*` (ex: `adtcore:name`, `adtcore:type`)
- Elementos aninhados `atom:content` → `abapProgram` / `abapClass`
- Pacote a partir do elemento filho `adtcore:packageRef`
- Usuário responsável a partir do atributo `adtcore:responsible`
- URI do código-fonte a partir de elementos `atom:link` (URIs relativas resolvidas para absolutas)

10. Ferramentas do Servidor MCP (21 ferramentas)

Categoria	Ferramenta	Descrição
Conexão	<code>sap_connect</code>	Autenticar no sistema SAP (suporta ambientes salvos via <code>system_id</code>)
Conexão	<code>sap_discovery</code>	Listar serviços ADT disponíveis
Credenciais	<code>sap_save_password</code>	Armazenar credenciais com segurança no gerenciador de credenciais do SO
Credenciais	<code>sap_delete_password</code>	Remover credenciais salvas
Credenciais	<code>sap_list_environments</code>	Listar ambientes SAP salvos (senhas nunca são exibidas)
Repositório	<code>sap_search</code>	Pesquisar objetos ABAP (curingas: <code>Z*</code> , <code>ZCL_*</code>)
Repositório	<code>sap_browse_package</code>	Listar conteúdo de pacotes (suporta pacotes com namespace)
Repositório	<code>sap_object_metadata</code>	Obter detalhes do objeto (nome, tipo, pacote, responsável, URI do fonte)
Código-Fonte	<code>sap_read_source</code>	Ler código-fonte ABAP (opcional: salvar em arquivo local)
Código-Fonte	<code>sap_write_source</code>	Gravar fonte + bloqueio automático + transport + ativação (de texto ou arquivo)
Código-Fonte	<code>sap_activate</code>	Compilar/ativar um objeto
Código-Fonte	<code>sap_syntax_check</code>	Verificação de sintaxe sem ativar
Transport	<code>sap_check_lock</code>	Verificar se o objeto está bloqueado e por quem
Transport	<code>sap_list_transports</code>	Listar transport requests abertos (modificáveis)
Transport	<code>sap_create_transport</code>	Criar novo transport request de workbench
Transport	<code>sap_release_transport</code>	Liberar um transport request
Transport	<code>sap_transport_check</code>	Verificar se objeto necessita de transport, listar transports disponíveis
Criação	<code>sap_create_program</code>	Criar novo programa ABAP (report)
Criação	<code>sap_create_class</code>	Criar nova classe ABAP global
Criação	<code>sap_create_interface</code>	Criar nova interface ABAP
Exclusão	<code>sap_delete_object</code>	Excluir um objeto ABAP (requer <code>confirm='DELETE'</code>)

Para detalhes completos dos parâmetros, exemplos e pré-requisitos SAP, consulte [MCP_COMMAND_REFERENCE.md](#).

11. Detalhes Técnicos Importantes

Sessões Stateful

O SAP ADT exige o header `X-sap-adt-sessiontype: stateful` nas operações de bloqueio, gravação e desbloqueio. Isso garante que o lock handle permaneça válido entre múltiplas requisições HTTP dentro da mesma sessão.

Headers Accept Específicos do Fornecedor

Muitos endpoints ADT rejeitam `application/xml` e exigem media types específicos do fornecedor (ex: `application/vnd.sap.adt.programs.programs.v2+xml`). O cliente seleciona automaticamente o header `Accept` correto com base na URI do objeto.

Tratamento de Pacotes com Namespace

Pacotes com namespace como `/TAX/SPED_MONITOR` requerem barras codificadas em percent-encoding nas URIs: `%2ftax%2fsped_monitor`. O método `browse_package` usa `POST` para `/sap/bc/adt/repository/nodestructure` com parâmetros de query (`parent_name`, `parent_tech_name`, `parent_type`, `withShortDescriptions`).

Parâmetros de Transport Request

O número do transport request (`corrNr`) e o lock handle (`lockHandle`) são enviados como **parâmetros de query** na requisição `PUT` , não como headers HTTP.

12. Instruções de Instalação

Pré-requisitos

- Python 3.10+
- Acesso de rede ao sistema SAP (ex: `awsntwpstx01.engdb.infra:8000`)
- Usuário SAP com autorização ADT (consulte [MCP_COMMAND_REFERENCE.md § 6](#))

Instalação

```
cd D:\ENGDB\sap-adt-python
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
```

Configuração Inicial de Credenciais

```
sap_save_password(
    system_id="ED2",
    host="awsntwpstx01.engdb.infra",
    port=8000,
    client="500",
    user="egoetz",
    password="****"
)
```

Teste de Conexão

```
python scripts\test_connection.py
```

Configuração MCP no Cursor

O workspace inclui `.cursor/mcp.json` com a chave de servidor `sap-adt` . No Cursor o servidor aparece como `user-sap-adt` .

Você pode copiar o mesmo bloco para `~/cursor/mcp.json` (global) ou ajustar os caminhos:

```
{
  "mcpServers": {
    "sap-adt": {
      "command": "python",
      "args": ["D:\\ENGDB\\sap-adt-python\\scripts\\run_mcp.py"],
      "env": {
        "SAP_HOST": "awsntwpstx01.engdb.infra",
        "SAP_PORT": "8000",
        "SAP_CLIENT": "500",
        "SAP_USER": "egoetz",
        "SAP_LANGUAGE": "EN"
      }
    }
  }
}
```

Com `sap_save_password` , o `SAP_PASSWORD` não é necessário na configuração — a senha é recuperada do gerenciador de credenciais do SO.

13. Dependências

Pacote	Versão	Finalidade
requests	>=2.31	Cliente HTTP para a API REST ADT
mcp	>=1.0	SDK do Model Context Protocol (FastMCP)
lxml	>=5.0	Parsing de XML para respostas ADT
keyring	>=25.0	Armazenamento seguro de credenciais no keyring do SO

14. Referências Técnicas

- API REST SAP ADT: Mesmos endpoints HTTP usados pelo plugin Eclipse ADT (`/sap/bc/adt/*`)
- Guia de Configuração SAP ADT: [Configuring the ABAP Back-end for ADT](#)
- SAP Development Tools: <https://tools.hana.ondemand.com/>
- Protocolo MCP: Model Context Protocol para integração de ferramentas com IA
- Biblioteca keyring: <https://pypi.org/project/keyring/>
- erpl-adt: Implementação CLI de referência (<https://github.com/DataZooDE/erpl-adt>)
- abap-adt-py: Biblioteca Python de referência (<https://pypi.org/project/abap-adt-py/>)

15. Documentação Relacionada

Documento	Descrição
README.md	Guia de início rápido
AGENTS.md	Instruções para agentes Cursor
MCP_COMMAND_REFERENCE.md	Referência completa de comandos com parâmetros, exemplos e pré-requisitos SAP
.cursor/rules/abap-dev.mdc	Regra do Cursor para workflow de desenvolvimento ABAP